Explain project requirements

10/17/2026 19:38:37
https://chatgpt.com/c/6ad41f78-57b0-83e8-b846-c347c444991c

cs454-fa26-project1(2).pdf Here is my project PDF can you explain to m…

Thinking ● Refining project requirements I’m identifying likely…

Why would remembering only the last five characters be enough to…

The reason we only need to remember the last five characters is becau…

How would I approach designing a DFA that checks every six-character…

To design a DFA that checks every six-character window, I would…

Can you walk me through a small example of the counting algorithm…

Thinking ● Counting DFA paths I’m illustrating the recurrence with a…

Why does the second DFA have 16 input symbols instead of four

Thinking ● Clarifying the construction I’m interpreting strings as \(w=x…

so should I use two DFA's?

Yes! Your CS 454 project requires you to construct two DFAs. However,…

sorry i mean should I use two copies of the first dfa to help with the…

Thinking ● Checking DFA construction I’m refining the product…

How can the second DFA determine whether joining two valid halves…

Thinking ● Designing the boundary check I’m describing a two-copy DF…

The second DFA we made runs really slow for n=300 why is that?

Thinking ● Assessing computational scale I’m estimating the workload:…

what should I do to speed up the seond DFA

Thinking ● Optimizing the recurrence I’m focusing on reducing each…

Selected text: ## Selection 1 Rather than counting every possible…

Thinking ● Clarifying the faster method I’m formulating the exact…

cs454-fa26-project1(2).pdf

Here is my project PDF can you explain to me what this project is asking for me to accomplish and please don't start writing any of the code

Thinking

- **Refining project requirements**

  I'm identifying likely ambiguities before explaining the solution: the second example has odd length despite requiring even length, and "three letters" should likely mean four. I'll also clarify state counts, length handling, and how dynamic programming avoids code.

- **Refined project requirements**

✓ Worked for 11s

Your **CS 454 Theory of Computation – Project 1** is asking you to build a program that uses **Deterministic Finite Automata (DFAs)** to count how many strings satisfy certain rules.

The project has two main problems, and you need to use an efficient counting algorithm rather than generating every possible string.

I'll break down what your professor expects, explain the concepts, and walk through how the pieces fit together. **I won't write any code.**

1. Understanding the overall goal

Your program works with strings made up of four letters:

$$\Sigma = \{a, b, c, d\}$$

You need to count strings that satisfy two different sets of conditions.

Problem 1 — Language L Menu option 1

Count all strings of length n where **every substring of length 6 contains all four letters**: a, b, c, and d.

Problem 2 — Language L′ Menu option 2

Count strings that follow the rules of Problem 1, but also have **even length and** `aa` **as their two middle characters**.

For both problems, your program must accept lengths between 1 and 300.

2. Problem 1: Understanding the language L

Your professor defines:

$$L = \{w \mid \text{every length-6 substring contains } a, b, c, d\}$$

Let's use some examples.

Valid string: `abbccd`

a b b c c d

Contains a, b, c, and d — accepted.

Invalid string: `aabbcc`

a a b b c c

Missing d — rejected.

If a string has 10 characters, you must check every consecutive group of six characters, not just its first six.

One important detail from the PDF: **all strings shorter than six characters are automatically accepted.** There isn't a complete six-character substring to violate the rule.

3. Your first major task: Construct a DFA

This is where the Theory of Computation concepts come in.

Your professor wants a DFA that recognizes language L.

You know from class that a DFA is defined using:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

Component	Meaning
Q	Set of states
Σ	Input alphabet
δ	Transition function
q_0	Starting state
F	Accepting states

For this project, you're not expected to manually draw a DFA with over a thousand states. **Your program must construct the DFA automatically.**

How will the DFA remember the characters?

Your professor suggests that each state represents a buffer containing the last five characters read.

Why five?

Because when you read the next character, you have six characters to examine.

Here's an interactive example of how the buffer works.

DFA buffer demonstration Characters read: 5/9

a b c d a **b** c d a

Current buffer (last five characters)

a b c d a + **b**

Valid six-character window

All four letters appear. Drop the oldest buffer character and append the new one.

‹ Previous Next character ›

Your DFA has to do three important things:

1. Remember the last five characters.
2. When a sixth character becomes available, check whether the resulting six-character window contains all four required letters.
3. If the condition fails, transition to a special **fail state**, from which it can never escape.

Your professor estimates that the DFA for L will have around 1,400 states.

The assignment also discusses using a base-4 encoding to assign numeric labels to states. This matters because your states need to be numbered 0, 1, 2, and so on.

4. Your second major task: Count the strings accepted by your DFA

This is probably the most important distinction in your project.

You are not being asked to generate every possible string and individually check whether it is accepted.

Instead, you're building an algorithm that calculates how many strings of a particular length a DFA accepts.

Why is this necessary?

There are four possible letters in each position, so the number of possible strings of length n is:

$$4^n$$

For length 300, that's an enormous number of possible strings.

Checking every string individually would be impractical.

Your professor wants you to use **dynamic programming**.

Understanding the counting algorithm

Your assignment defines:

$$N_j(n)$$

This represents the number of strings of length n that, starting from state j, end in an accepting state.

The recurrence is:

$$N_j(n) = \sum_{x \in \{a,b,c,d\}} N_{\delta(j,x)}(n-1)$$

In simpler terms:

You calculate how many ways you can reach an accepting state by taking one transition and then completing the remaining characters.

Your professor specifically wants you to use two arrays:

prev	**next**
Stores counts for the current length	Stores counts for the next length

↓↑

Calculate the next counts from the previous counts, then copy the new values back. Repeat until you reach length n.

The important part is that each array has one entry per DFA state.

You don't need to store all possible strings or a complete history of counts for every length.

This is the efficiency improvement your professor expects you to implement.

5. Your third major task: Construct a second DFA for L″

This is where the second counting problem becomes more complicated.

The second language is:

$$L' = \{w \in L \mid |w| \text{ is even and the middle two symbols are } aa\}$$

For example:

`bcd` + `aa` + `bcd`

The complete string is `bcdaabcd`, which has eight characters. Its middle two characters are `aa`, and every six-character substring contains a, b, c, and d.

However, your professor explains that **L′ is not a regular language**.

Therefore, you can't directly construct an ordinary DFA recognizing all of L′.

Instead, your professor wants you to build a DFA for another language, called L″.

The PDF specifies that:

$$\#L'_{2m} = \#L''_{m-1}$$

This means the number of strings of length $2m$ in L′ is equal to the number of strings of length $m - 1$ accepted by the DFA for L″.

Your professor says the DFA for L″ will have approximately 1400^2 states and use an alphabet of 16 symbols, numbered 0 through 15.

Important: The PDF does not explain the complete construction of L″. It explicitly states that those details will be described in class.

So this is something you'll need to understand from your lecture notes before implementing the second DFA.

6. Your fourth major task: Create the user interface

Your professor specifies a simple menu-based interface.

It should behave something like this:

Example of the program's interaction (illustrative only)

```
1. Count strings in L
2. Count strings in L'
3. Quit
```

Choose an option

- 1 — Count strings in L
- 2 — Count strings in L′
- 3 — Quit

Enter n (1–300)

Unsupported component: input

The program calls the appropriate counting operation and displays the exact result.

This is only a demonstration of how the interaction could look, not code or a prescribed interface design.

Your program must continue showing the menu after either counting operation and only terminate when the user chooses option 3.

The assignment requires exact integer answers, including for $n = 300$. Python's integers and Java's `BigInteger` are examples of ways to support large exact counts.

7. What exactly do you need to turn in?

According to page 2 of your PDF:

Deliverable	Requirements
Program	Both DFA constructions and the counting functionality
Executable	Include the compiled executable on the department server, along with all required files
AI transcript	Submit the full conversation with any AI assistant you used
Report	One page explaining implementation details and pseudocode of the main algorithms
Test examples	Sample inputs and outputs, including $n = 300$

You also need to identify each team member's contribution. Only one submission is required per team.

Your professor explicitly states that **the report must be completed without any support from an AI assistant**. I can help you learn and understand the concepts for the project, but the actual report and its pseudocode need to be your own work.

8. The order I would recommend tackling the project

I would approach it in this order so that each step builds on the previous one.

Project checklist 0 of 8 completed

- [ ] Understand what makes a string valid in L
- [ ] Understand how the five-character buffer represents a DFA state
- [ ] Work out how DFA states and transitions are created
- [ ] Understand the dynamic programming counting recurrence
- [ ] Understand the second DFA, L″, using your class notes

- [ ] Implement and test both counting options and the menu
- [ ] Verify test cases, including n = 300
- [ ] Independently prepare the required report and submission

The main thing I want you to understand

This isn't really a project about checking whether individual strings are valid. It's about combining two things you've learned in Theory of Computation:

DFAs — recognizing strings that follow a set of rules.

Dynamic programming — efficiently counting the strings those DFAs accept.

Once you understand how to construct a DFA for L and count its accepted strings, you've built the foundation for much of the project.

My recommendation: Start by making sure you understand why the DFA only needs to remember the last five characters and how a state transitions when it reads another character. That's the first concept I would focus on before trying to implement anything.

gpt-6-thinking

Why would remembering only the last five characters be enough to determine whether a string is valid?

The reason we only need to remember the **last five characters** is because your project only cares about whether **every substring of length 6 contains all four letters: a, b, c, and d.**

When a new character is added, we only need to check the newest group of six characters.

Let's walk through an example.

1. Start with five characters

Suppose our DFA has read:

`abcda`

We haven't checked any six-character substrings yet because we only have five characters.

Our DFA remembers:

a b c d a

The DFA's five-character buffer

2. Read the next character

Suppose the next character is `b`.

Now we have:

a b c d a b

Valid: contains a, b, c, and d.

Since the substring is valid, we can continue reading.

3. Forget the oldest character

Now comes the important part.

We no longer need to remember the first `a`.

Why?

Because **that first `a` will never be part of another six-character substring formed by adding characters to the end.**

We already checked the only six-character window containing that first character.

So our buffer changes:



The updated five-character buffer is `bcdab`.

If another character arrives, such as `c`, the DFA checks `bcdabc`.

Then it forgets the oldest `b` and remembers `cdabc`.

This process repeats for every character.

4. But what if an earlier substring was invalid?

This is an important question.

Suppose the DFA reads:

`aabbcc`

This is invalid because it doesn't contain `d`.

The DFA immediately enters a **fail state**.

Once in the fail state, it stays there regardless of what characters arrive next.

Even if a `d` appears later, the original invalid substring still exists, so the entire string must be rejected.

This is how the DFA remembers whether an earlier part of the string violated the rule without storing the entire string.

5. The key idea

The DFA only needs two pieces of information:

1. **The last five characters**, so it can check the next six-character substring.

2. **Whether a violation has already occurred**, represented by the fail state.

It doesn't need to remember older characters because any six-character substring involving them has already been checked.

Quick question to test your understanding

Suppose the DFA currently remembers:

`abbcd`

And the next character is `a`.

What should happen?

- A. Transition to the fail state
- B. Accept the new character and update the buffer to `bbcda`
- C. Keep the buffer as `abbcd`
- D. Remember all six characters

Check answer

gpt-6-thinking

How would I approach designing a DFA that checks every six-character window of a string?

To design a DFA that checks every six-character window, I would approach it by breaking the problem into smaller pieces rather than trying to build the entire DFA at once.

For your CS 454 project, the goal is to construct a DFA that recognizes:

$$L = \{w \mid \text{every substring of length } 6 \text{ contains } a, b, c, d\}$$

The important thing to understand is that **you don't need a separate DFA for every six-character substring.** You need one DFA that continuously tracks enough information to check each new window.

Step 1: Decide what your states represent

Normally, when designing a DFA, you start by asking:

What information do I need to remember about the input I've read so far?

For this project, that information is the last five characters.

Instead of creating arbitrary states named q_0, q_1, q_2, you can think of states as strings:

- `ε` — Nothing has been read yet.
- `a` — The last character was a.
- `ab` — The last two characters were ab.
- `abc` — The last three characters were abc.
- `abcd` — The last four characters were abcd.
- `abcda` — The last five characters were abcda.

These states represent the DFA's memory.

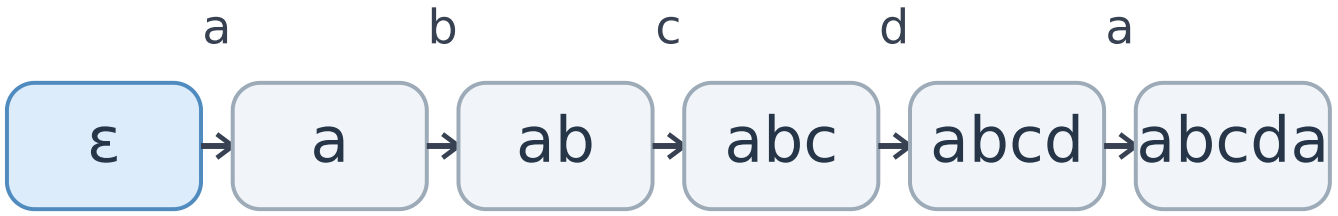
Your actual implementation will eventually need to assign numeric IDs to them, as required by the assignment.

Step 2: Design the transitions for the first five characters

When the DFA starts, it hasn't read anything.

Each new character gets added to the state until the buffer contains five characters.

For example:



A single example path through the DFA, not the complete transition diagram.

No six-character window exists yet, so nothing can violate the condition.

All these states must be accepting states, because the assignment specifies that every string of length less than six belongs to L.

Step 3: Design transitions once the buffer is full

This is where you begin checking the actual condition.

Suppose you're in state:

`abcda`

And the DFA reads `b`.

You combine the current buffer with the new symbol:

`abcda` + `b` = `abcdab`

Now ask:

Does `abcdab` *contain all four letters?*

Yes.

Therefore, the transition goes to the state representing the last five characters:

$$\delta(\text{abcda}, b) = \text{bcdab}$$

If you read a different character, you perform the same check.

For example:

$$\delta(\text{abcda}, a) = \text{bcdaa}$$

This transition is also valid because `abcdaa` contains all four letters.

Step 4: Create a rejecting fail state

Now consider a different situation.

Current state: `aabbc`

New character: `a`

The resulting six-character window is:

`aabbca`

It contains a, b, and c, but **no d**.

Therefore, the DFA must reject the string.

Rather than tracking anything else, you transition to a special fail state.

$$\delta(\text{aabbc}, a) = q_{\text{fail}}$$

Once the DFA enters this state, every future input keeps it there:

$$\delta(q_{\text{fail}}, x) = q_{\text{fail}}$$

for every $x \in \{a, b, c, d\}$.

This ensures that an invalid substring cannot be fixed by adding more characters.

Step 5: Understand how every window gets checked

Here is an example using the string `abcdabcd`.

Sliding window demonstration Window 1 of 3

a	b	c	d	a	b	c	d

Checking: **abcdab** Valid

‹ Previous Next window ›

Notice that every window overlaps the previous window by five characters.

That's exactly why the DFA's buffer holds five characters.

Every time another character is read, a new window can be checked.

Step 6: Think about how to build all the states

Once you understand the transition rules, the next challenge is creating the entire DFA.

Your professor expects approximately 1,400 states, so manually defining them isn't realistic.

Conceptually, you will need to:

1. Establish the initial state representing the empty string.
2. Represent the possible buffers containing one through five characters.
3. Determine the transition from each state for all four possible input symbols.
4. Send any invalid six-character window to the fail state.
5. Label the states with integer IDs, as your professor requires.

You should also think about which states can actually be reached from the starting state and which ones are accepting.

The PDF suggests using **base-4 encoding** to help represent the buffer states numerically.

Step 7: Test your understanding before implementing anything

Here are three cases worth working through on paper.

For each case, decide which state the DFA transitions to.

Question 1

Current state: `{q.buffer}` | New character: `{q.incoming}`

- ○ State bcdac
- ○ Fail state
- ○ State abcdac

Question 2

Current state: `{q.buffer}` | New character: `{q.incoming}`

- State aabcd
- Fail state
- State abbcd

Question 3

Current state: `{q.buffer}` | New character: `{q.incoming}`

- State bccab
- Fail state
- State bbcca

The next concept I'd recommend studying is how to assign numeric IDs to these buffer states. Understanding that will help you bridge the gap between a DFA you can describe on paper and one your program can construct automatically, without having to manually define thousands of states.

gpt-6-thinking

Can you walk me through a small example of the counting algorithm using a simple DFA with only a few states?

Thinking

- **Counting DFA paths**

  I'm illustrating the recurrence with a three-state DFA recognizing strings containing "aa." I'll define transitions, initialize length zero, update previous and next counts, and verify accepted strings at length three.

- **Counted DFA paths**

✓ Worked for 8s

Absolutely! Let's walk through a much smaller version of the counting algorithm from your CS 454 project.

We'll use a **3-state DFA**, calculate how many strings of different lengths it accepts, and then connect that back to your project's `prev` and `next` arrays.

No code yet — just the logic and math behind the algorithm.

Step 1: Create a simple DFA

Instead of using your project's alphabet `{a, b, c, d}`, let's use:

$$\Sigma = \{a, b\}$$

Our DFA will recognize strings that **contain** `aa` **somewhere in the string**.

For example:

- `aab` → Accepted
- `baa` → Accepted
- `aba` → Rejected
- `bbb` → Rejected

We'll need three states:

State	Meaning	Accepting?
q_0	Haven't found `aa`, and the last character isn't `a`	No
q_1	Haven't found `aa`, but the last character is `a`	No
q_2	Already found `aa`	Yes

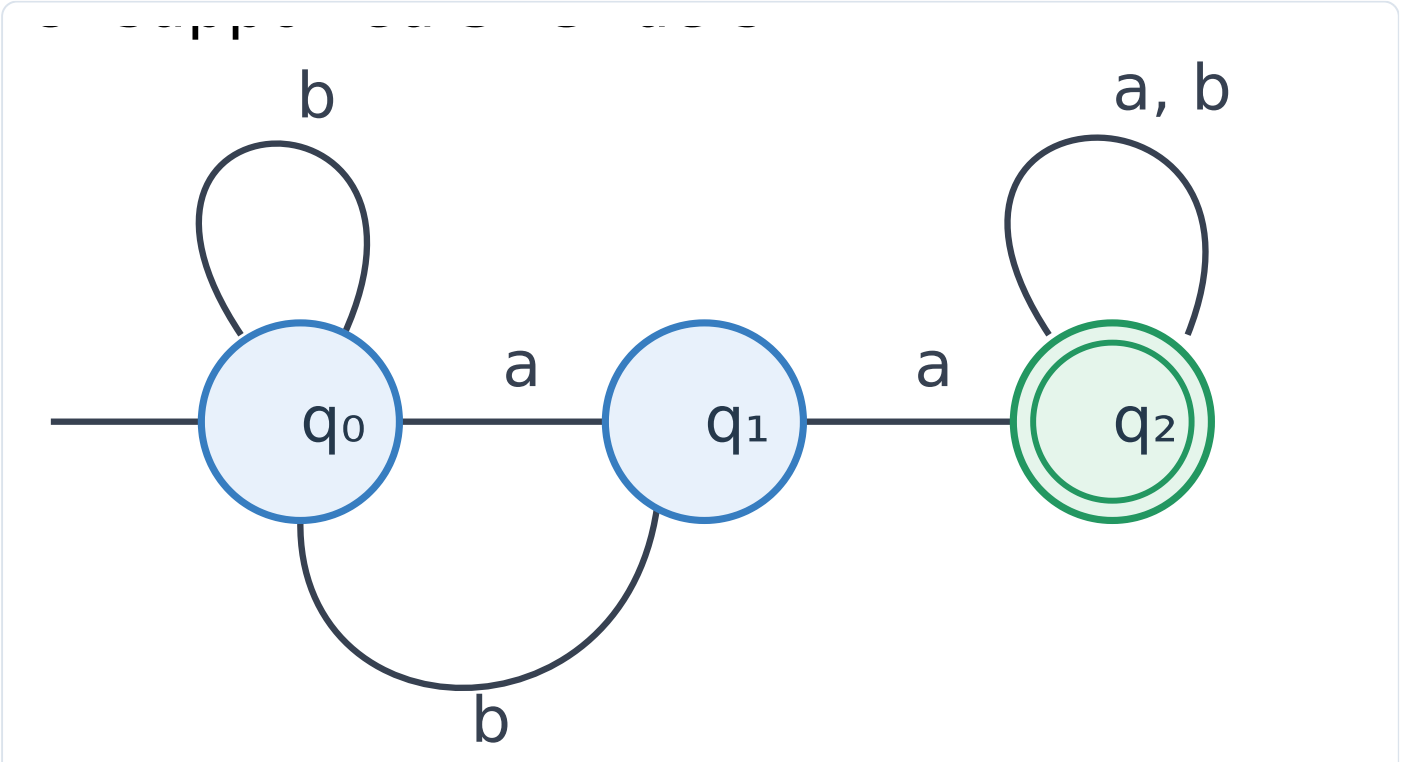
Our starting state is q_0.

Step 2: Define the transitions

Here is the transition table:

Current state	Input `a`	Input `b`
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_2

And here is what that DFA looks like:



The double circle means q_2 is an accepting state.

Once we reach q_2, we've already found `aa`, so it doesn't matter what characters come afterward.

Step 3: Understand what we're counting

Your professor defines:

$$N_j(n) = \text{number of accepted strings of length } n \text{ starting at state } j$$

Here is the critical detail:

We must calculate counts starting from every state, not just the starting state.

Why? Because calculating the counts for q_0 depends on the counts for other states.

The recurrence from your assignment is:

$$N_j(n) = \sum_{x \in \Sigma} N_{\delta(j,x)}(n-1)$$

For our simplified DFA, the alphabet contains only `a` and `b`.

Therefore:

$$N_j(n) = N_{\delta(j,a)}(n-1) + N_{\delta(j,b)}(n-1)$$

In other words, the number of accepted strings of length n is the number of ways to finish after reading `a`, plus the number of ways to finish after reading `b`.

Step 4: Start with strings of length 0

This is called the **base case**.

Your professor gives the rule:

$$N_j(0) = \begin{cases} 1 & \text{if state } j \text{ is accepting} \\ 0 & \text{otherwise} \end{cases}$$

Why is this true?

Because there is exactly one string of length zero: the empty string, ϵ.

If you're already in an accepting state, that empty string is accepted.

So our initial counts are:

State	$N_j(0)$
q_0	0
q_1	0
q_2	1

Our `prev` array initially represents:

$$prev = [0, 0, 1]$$

Step 5: Calculate strings of length 1

Now we use the recurrence formula.

For q_0, reading `a` takes us to q_1, while reading `b` keeps us at q_0.

Therefore:

$$N_0(1) = N_1(0) + N_0(0) = 0 + 0 = 0$$

For q_1:

$$N_1(1) = N_2(0) + N_0(0) = 1 + 0 = 1$$

For q_2:

$$N_2(1) = N_2(0) + N_2(0) = 1 + 1 = 2$$

So the new counts are:

$$next = [0, 1, 2]$$

Then, as your professor requires, copy `next` into `prev`.

Step 6: Repeat for lengths 2 and 3

Now we repeat the same process, using the counts from the previous length.

For length 2:

$$N_0(2) = N_1(1) + N_0(1) = 1 + 0 = 1$$
$$N_1(2) = N_2(1) + N_0(1) = 2 + 0 = 2$$
$$N_2(2) = N_2(1) + N_2(1) = 2 + 2 = 4$$

And then length 3:

$$N_0(3) = N_1(2) + N_0(2) = 2 + 1 = 3$$
$$N_1(3) = N_2(2) + N_0(2) = 4 + 1 = 5$$
$$N_2(3) = N_2(2) + N_2(2) = 4 + 4 = 8$$

Here is the complete table:

String length	$N_0(n)$	$N_1(n)$	$N_2(n)$
0	0	0	1
1	0	1	2
2	1	2	4
3	**3**	5	8
4	8	13	16

The answer for strings of length 3 is **3**, because q_0 is our starting state.

Let's verify that manually.

There are eight possible strings of length three over `{a,b}`:

String	**Contains `aa`?**
`aaa`	✓
`aab`	✓
`aba`	×
`abb`	×
`baa`	✓
`bab`	×
`bba`	×
`bbb`	×

Exactly three strings are accepted, matching our counting algorithm.

Step 7: Try the counting algorithm yourself

Use this interactive example to advance the counts one string length at a time.

Counting algorithm simulator

n = 0

DFA with 3 states and 2 input symbols

Current `prev` array

q0	q1	q2
0	**0**	**1**

Calculated `next` array

q0	q1	q2
0	**1**	**2**

Accepted strings of length 0:

0

We always use the count for q_0 because it is the DFA's start state.

‹ Previous Next length ›

Step 8: Connect this to your CS 454 project

Now imagine replacing our small 3-state DFA with the roughly 1,400-state DFA your professor describes.

The exact same process applies:

1. Build your DFA and its transition table.
2. Create two arrays, `prev` and `next`, each with one entry per state.
3. Initialize `prev` to 1 for accepting states and 0 for rejecting states.
4. For every state, calculate the number of accepted strings of the next length using its transitions.
5. Copy the new counts into `prev` and repeat until you reach the requested length n.
6. Return the value for the starting state (state 0).

The main difference is that your actual DFA has **four possible transitions per state**, rather than two.

That means your formula becomes:

$$\boxed{N_j(n) = N_{\delta(j,a)}(n-1) + N_{\delta(j,b)}(n-1) + N_{\delta(j,c)}(n-1) + N_{\delta(j,d)}(n-1)}$$

This is how you can count valid strings of length 300 without generating any of those strings.

Quick understanding check

Suppose we're using our example DFA, and our arrays currently contain:

$$prev = [1, 2, 4]$$

This represents the counts for strings of length 2.

What would the new value of `next[0]` be?

Remember, from q_0:

- Reading `a` takes us to q_1.
- Reading `b` takes us to q_0.

- 1
- 2
- 3
- 6

If this makes sense, you've understood the main counting algorithm your professor wants you to use.

gpt-6-thinking

Why does the second DFA have 16 input symbols instead of four

Thinking

- **Clarifying the construction**

  I'm interpreting strings as \(w=xaa y\), with equal outer lengths. Symmetric pairs are processed from the center outward, using two copies of the automaton; the exact \(L''\) details remain inferred because the source defers them to class.

- **Refining the pairing method**

  I'm treating each input pair as one symbol from the left and one from the right. Two automaton states track inward scans, while final buffers verify windows crossing the central \(aa\).

- **Clarified the construction and refined the pairing method**

✓ Worked for 13s

The second DFA uses **16 input symbols instead of 4 because each input symbol represents a pair of characters rather than a single character.**

This is related to the second problem in your CS 454 project, where you're counting valid strings that have `aa` in the middle.

Let's break down why this works.

1. Remember what the second problem requires

Your second language is:

$$L' = \{w \in L \mid |w| \text{ is even, and the middle two characters are } aa\}$$

For example, consider:

`bcdaabcd`



Even length, middle `aa`, and every six-character window is valid.

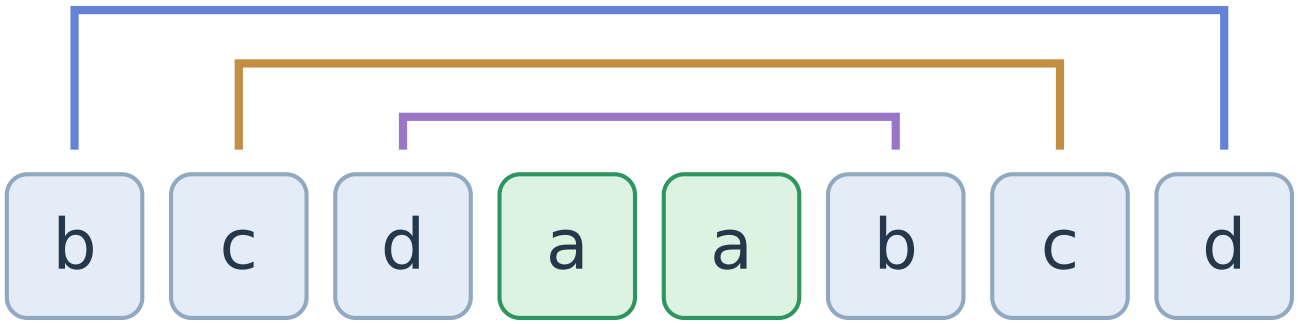
The important observation is that the middle `aa` is fixed.

Therefore, we can think about the characters on either side of it.

2. Think of the string as pairs of characters

One possible way to understand the second DFA is to process a character from the left side and a character from the right side together, moving inward.

For our example:



Fixed middle: aa

3 pairs processed from outside inward

First pair	`(b, d)`
Second pair	`(c, c)`
Third pair	`(d, b)`

Instead of processing eight individual characters, we have three pairs and the fixed middle `aa`.

3. Why does that give us 16 symbols?

Each character in a pair can be one of four possibilities:

$$\Sigma = \{a, b, c, d\}$$

The first character has 4 possibilities.

The second character also has 4 possibilities.

Therefore:

$$|\Sigma''| = 4 \times 4 = \boxed{16}$$

Here are all 16 possible pairs:

aa 0	ab 1	ac 2	ad 3
ba 4	bb 5	bc 6	bd 7
ca 8	cb 9	cc 10	cd 11
da 12	db 13	dc 14	dd 15

One possible numeric encoding of the 16 input symbols, from 0 to 15.

This is why your professor says the second DFA uses input symbols labeled 0 through 15.

Each number represents one of these 16 possible pairs.

4. How does this relate to the length of the string?

Your professor provides the relationship:

$$\#L'_{2m} = \#L''_{m-1}$$

Let's understand that using a string of length 8.

$$2m = 8$$

Therefore:

$$m = 4$$

And:

$$m - 1 = 3$$

So instead of counting strings of length 8 directly, you count accepted strings of **3 paired symbols** in L″.

Each paired symbol represents two original characters, and the two middle characters are always `aa`.

That accounts for:

$$3(2) + 2 = 8$$

5. Why does the second DFA have so many states?

Your professor says the first DFA has approximately 1,400 states, but the second has approximately:

$$1400^2 = 1,960,000$$

This suggests using a **product construction** involving two DFA states.

Conceptually, a state in the second DFA could track two states from the original DFA:

$$(q_i, q_j)$$

One component tracks information about one side of the string, and the other tracks the opposite side.

When reading an input pair, both components can be updated.

There is an important additional detail: the automaton must also check the six-character windows that cross the fixed middle `aa`. Checking the two halves independently would not be sufficient.

Your PDF doesn't specify the exact transitions or accepting-state conditions for this product construction; it says those details will be explained in class. The outside-in pairing above is a way to understand the 16-symbol alphabet, not a claim about the precise construction your professor expects.

---

6. The main differences between the two DFAs

Feature	First DFA (L)	Second DFA (L″)
Input represents	One character	Two characters
Alphabet size	4	16
Approximate states	1,400	1,960,000

Feature	First DFA (L)	Second DFA (L″)
Purpose	Enforce every six-character window	Help count valid strings with middle `aa`
Counting length	n	$m-1$, where $n=2m$

The key takeaway: The second DFA has 16 input symbols because it's effectively considering two character choices simultaneously. Since each character can be a, b, c, or d, there are $4^2 = 16$ possible combinations.

And here's something important to consider next: **If the second DFA tracks two states from the first DFA, why would we need both states rather than just remembering the last five characters like before?** That's the question that will help you understand the product DFA construction.

gpt-6-thinking

so should I use two DFA's?

Yes! Your CS 454 project requires you to construct two DFAs. However, you only need **one counting algorithm** that works with either DFA.

Here's how I would organize the project conceptually.

1. DFA #1 — Language L

This DFA handles the first problem.

- **Alphabet:** `{a, b, c, d}` (4 symbols)
- **States:** Approximately 1,400
- **Purpose:** Check that every six-character substring contains all four letters.
- **How it works:** Remembers the last five characters and transitions to a fail state if a six-character window is invalid.

2. DFA #2 — Language L″

This DFA handles the second problem.

- **Alphabet:** 16 symbols (representing pairs of characters)
- **States:** Approximately 1400^2
- **Purpose:** Help count valid even-length strings with `aa` in the middle.
- **How it works:** Uses a construction related to the first DFA, tracking pairs of states.

Remember, your professor says the exact construction of L″ will be explained in class.

3. One shared counting algorithm

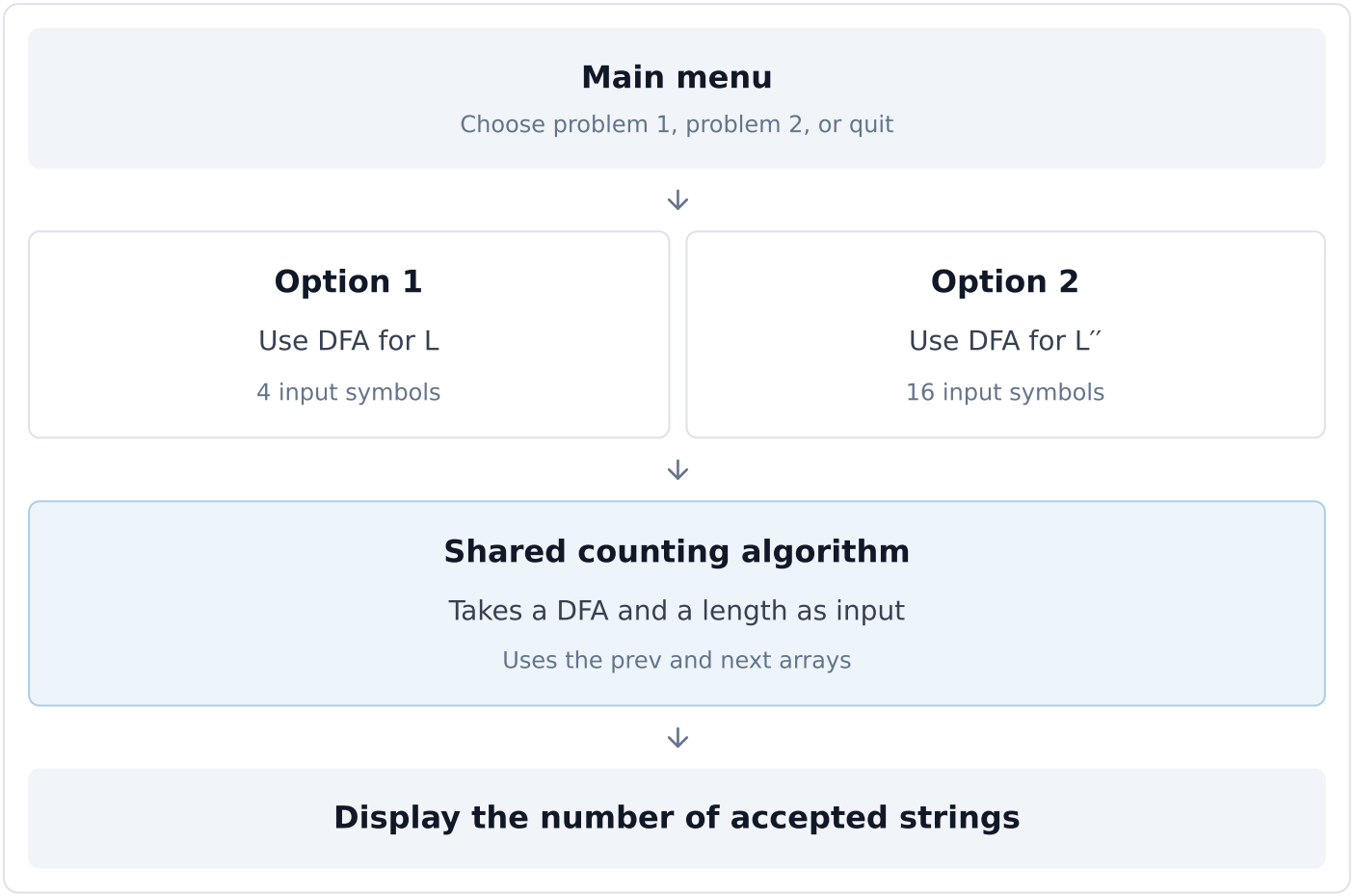
You don't need to design a different counting algorithm for each DFA.

Both use the same recurrence:

$$N_j(n) = \sum_{x \in \Sigma} N_{\delta(j,x)}(n-1)$$

The difference is that DFA #1 has 4 possible transitions per state, while DFA #2 has 16.

Here's how the program would be organized:



4. What I recommend you do first

Start by understanding and designing **DFA #1 completely** before worrying about DFA #2.

The first DFA will teach you how to represent states, construct transitions, handle invalid strings, and use the counting algorithm.

Once that works, you'll have a foundation for the second DFA.

One important distinction: You need two DFA constructions, but that doesn't mean you need two completely separate implementations of the counting algorithm.

And since you're still in the design phase, here's the next question I'd recommend thinking about:

How can you represent the states and transitions of DFA #1 so that your counting algorithm can access them efficiently?

That is the connection between understanding the DFA mathematically and eventually implementing your project.

gpt-6-thinking

sorry i mean should I use two copies of the first dfa to help with the second dfa I need to make?

Thinking

- **Checking DFA construction**

  I'm refining the product construction: process matching symbols from both ends, use one DFA forward and another on the reversed right half, retain five-symbol boundary memory, and verify all windows crossing the central `aa`, including short cases.

- **Finalizing product details**

  I'm representing each paired input symbol by a product state \((p,q)\): one DFA copy scans the left half forward, the other scans the right half backward. Acceptance checks internal validity and every window crossing the central `aa`.

- **Checked DFA construction and finalized product details**

✓ Worked for 12s

Yes! Using two copies of your first DFA is a good way to approach constructing the second DFA (L″). This is called a **product DFA construction**.

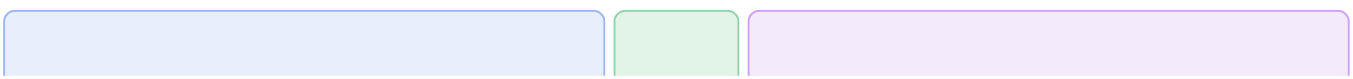
Your professor's PDF supports this approach because the second DFA has approximately 1400^2 states, although the exact construction is left for discussion in class.

1. Why use two copies?

Remember, your second problem requires:

- Every six-character substring must contain a, b, c, and d.
- The string must have even length.
- The middle two characters must be `aa`.

We can split a valid string into three parts:



bcd	aa	bcd
Left half	Middle	Right half

The idea is that you can use:

DFA #1 (copy A): Processes characters from the left side, moving toward the middle.

DFA #1 (copy B): Processes characters from the right side, moving toward the middle.

Because your language's six-character condition works the same forward and backward, the original DFA's transition rules can be used for both scanning directions.

2. How would the two DFAs work together?

Consider the string `bcdaabcd`.

Step	Copy A reads	Copy B reads
1	b	d
2	c	c
3	d	b

Each step reads a pair of characters.

That's where your **16 input symbols** come from:

$$4 \times 4 = 16$$

The product DFA tracks the current state of both copies simultaneously.

If copy A is in state q_5 and copy B is in state q_{12}, the combined DFA state is:

$$(q_5, q_{12})$$

If each original DFA has approximately 1,400 states, the product could have:

$$1400 \times 1400 = 1,960,000$$

possible paired states.

3. There's one important complication

You cannot simply have both copies check their respective halves and assume the entire string is valid.

You must also check the six-character windows that cross the middle `aa`.

For example, consider:

`abc` + `aa` + `abc`

Each side is shorter than six characters, so neither individual DFA would reject its half.

But the complete string `abcaaabc` is invalid because its six-character windows do not contain `d`.

This means your second DFA needs to use the information from both component states to decide whether the characters surrounding the middle `aa` satisfy the condition.

In the natural product construction, this can be handled through the accepting-state conditions, because the component states remember the characters closest to the middle.

4. What this means for your project

You would conceptually have:

Component	Responsibility
Original DFA M	Recognize the six-character condition
Copy A	Track the left half
Copy B	Track the right half in reverse
Product DFA L″	Track both copies and enforce the middle boundary condition
Counting algorithm	Count accepted strings using either DFA

You aren't necessarily creating two entirely separate copies in memory. You are using pairs of states and the original transition rules to describe a larger DFA.

So your understanding is correct: The second DFA can be constructed using two instances of the first DFA working together.

One question worth thinking about

Suppose copy A has remembered the five characters closest to the middle on the left, and copy B has remembered the five characters closest to the middle on the right.

Would those two buffers, along with the fixed `aa`, give you enough information to check every six-character window that crosses the middle?

That is the next key concept to understand before designing your second DFA.

gpt-6-thinking

How can the second DFA determine whether joining two valid halves around aa creates an invalid six-character window

Thinking

- **Designing the boundary check**

  I'm describing a two-copy DFA construction: each half is checked while scanning inward, while short suffix buffers are retained. After the central `aa`, I test every length-six window spanning either boundary, including both sides when halves are short.

- **Explaining buffer necessity**

  I'm showing why valid halves don't guarantee a valid whole string. For example, `abcdd` and `abcdd` each pass individually, but joining them around `aa` creates `cddaaa`, a six-character window missing `b`; boundary buffers catch it.

- **Illustrating the boundary failure**

  I'm refining the example: `abcdab` and `bcabcd` each contain every required symbol in their six-character windows. Yet joining them with `aa` produces `dabaab`, which omits `c`; retaining five nearby symbols exposes this failure.

- **Explained boundary checks, buffer necessity, and failure cases**

✓ Worked for 25s

The second DFA can determine whether joining two valid halves around `aa` creates an invalid six-character window by **remembering the five characters closest to the middle from each half**.

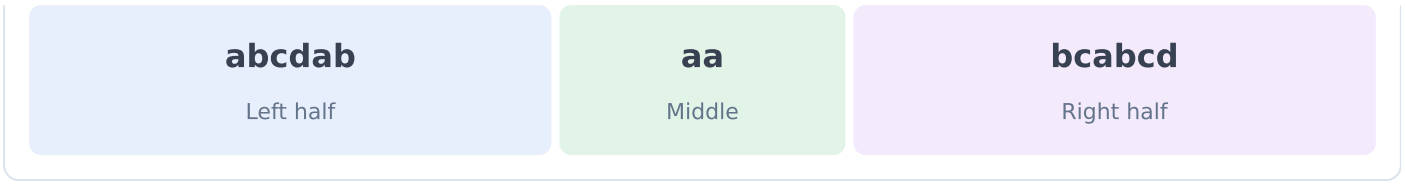
This is one of the main reasons using two copies of your first DFA is useful.

Let's walk through an example without writing any code.

1. Start with two individually valid halves

Consider the string:

`abcdab` + `aa` + `bcabcd`



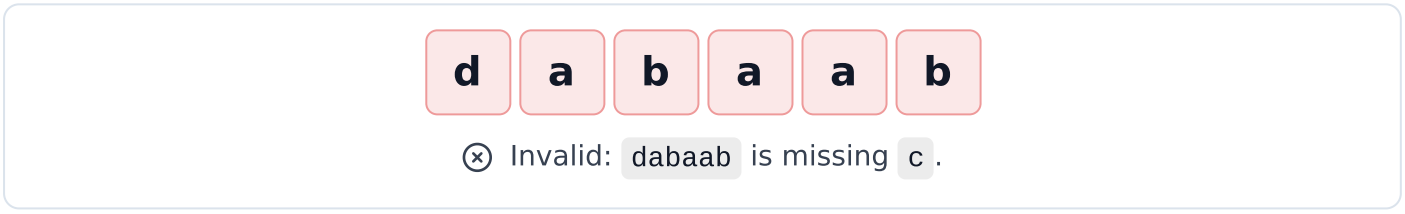
Both halves are valid by themselves:

- `abcdab` contains all four letters.
- `bcabcd` contains all four letters.

However, **the entire string is invalid**!

Why? Because when we join the two halves with `aa`, we create new six-character windows that weren't checked by either DFA.

One of these new windows is:



Even though both halves were valid, the complete string must be rejected.

2. How can the second DFA detect this?

Remember how your first DFA stores the last five characters it reads?

We can take advantage of that.

Copy A reads the left half normally, from left to right.

Copy B reads the right half backward, from right to left.

The six-character condition is unchanged when a string is reversed, so the same DFA transition rules can check both halves.

After processing the two halves, their buffers would be:

DFA	Characters processed	Final buffer
Copy A	`abcdab`	`bcdab`
Copy B	`dcbacb`	`cbacb`

Notice that Copy B stores its characters in reverse order.

To reconstruct the characters on the right side of the middle, we reverse Copy B's buffer:

`cbacb` → `bcabc`

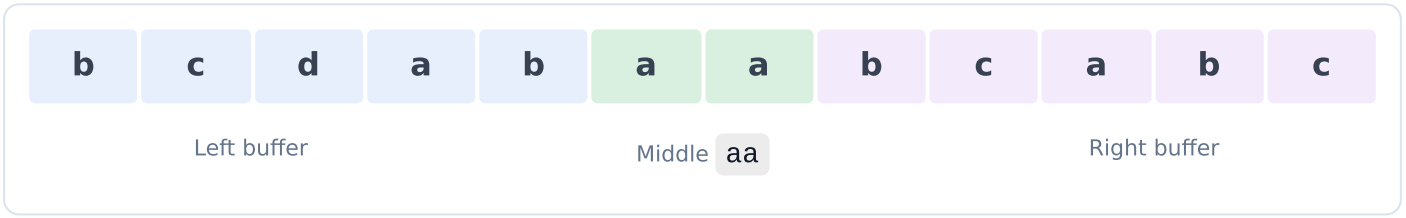
Now we have everything needed to examine the middle boundary.

3. Reconstruct the area around `aa`

We combine:

- Last five characters of the left half: `bcdab`
- Fixed middle: `aa`
- First five characters of the right half: `bcabc`

This gives us:



This reconstructed section contains every possible six-character window crossing the middle boundary.

Any window completely inside a half has already been checked by its DFA.

We only need to examine the windows involving the boundary.

4. How does the second DFA decide to accept or reject?

This is where the **accepting states of your product DFA** become important.

Remember that a product DFA state represents a pair:

$$(q_i, q_j)$$

Where q_i is the current state of Copy A and q_j is the current state of Copy B.

For a combined state to be accepting, three conditions must hold:

1. Copy A must be in an accepting state.
2. Copy B must be in an accepting state.
3. Every six-character window crossing the middle `aa` must contain all four letters.

If any condition fails, the combined state is not accepting.

This means the product DFA can use the information stored in its two component states to determine whether joining the halves creates an invalid window.

One useful detail: **the boundary check can be part of deciding which product states are accepting.** It doesn't necessarily require another DFA to check the middle.

This is a conceptual product construction consistent with your assignment; the PDF says the precise construction of L″ would be explained in class.

---

5. Why are five characters from each side enough?

Think about a six-character window crossing a boundary.

The most characters such a window could use from either side is five, because at least one character must come from the other side.

So anything farther than five positions from the middle cannot participate in a new six-character window crossing that middle region.

Those older characters have already been checked.

This is why the two five-character buffers, together with `aa`, give us all the information we need.

6. Check your understanding

Suppose your two halves are:

Left half: `abcdd`

Right half: `abcdd`

The complete string would be:

`abcddaaabcdd`

Both halves are shorter than six characters, so they are automatically valid individually.

Can the complete string be accepted?

- ○ Yes, because both halves are valid.
- ○ No, because a window crossing the middle is invalid.

The big takeaway: Your second DFA isn't just checking whether two halves are individually valid. It must also determine whether those halves are *compatible when connected by* `aa`.

That compatibility check is what allows the product DFA to recognize the paired representations of strings in L′.

gpt-6-thinking

The second DFA we made runs really slow for n=300 why is that?

Thinking

- **Assessing computational scale**

  I'm estimating the workload: a product automaton has about 1.96 million states and 31.36 million transitions per iteration. Across 149 iterations, eager construction or repeated nested loops can become extremely slow and memory-heavy, especially with object-heavy storage.

- **Assessed computational scale**

✓ Worked for 12s

The second DFA is probably running slowly because **it's dramatically larger than your first DFA**, and the counting algorithm has to perform billions of calculations for large inputs.

This is actually one of the biggest challenges in your CS 454 project.

1. Compare the size of your two DFAs

Your professor estimates:

	First DFA (L)	Second DFA (L″)
Number of states	~1,400	~1,960,000
Input symbols	4	16
Transitions per iteration	5,600	31,360,000

Because your second DFA uses pairs of states from the first DFA, the number of possible states grows approximately as:

$$1400^2 = 1,960,000$$

And each state has 16 possible input symbols instead of 4.

That means **one iteration of your counting algorithm could require more than 31 million transition lookups and additions.**

2. What happens when n = 300?

Remember the relationship your professor gives:

$$\#L'_{2m} = \#L''_{m-1}$$

For an original string length of 300:

$$2m = 300 \Rightarrow m = 150$$

Therefore, your second DFA should count strings of length:

$$m - 1 = \boxed{149}$$

So your counting algorithm only needs 149 iterations for the second DFA.

However, that's still an enormous amount of work.

Estimated transition evaluations for n = 300

4.67 billion

1,960,000 states × 16 symbols × 149 iterations

This is assuming your program examines every state and every transition during every iteration.

3. Why Python might struggle

If you're using Python, several things could be slowing your implementation down.

First: The nested loops. Iterating over millions of states and 16 transitions per state, hundreds of times, becomes expensive in Python.

Second: Memory usage. If you explicitly store 16 transitions for approximately 1.96 million states, that's over 31 million transition entries. Python's data structures can consume hundreds of megabytes or more storing them.

Third: Recalculating transitions. If your counting algorithm rebuilds transition information repeatedly instead of reusing it, that adds substantial overhead.

Fourth: Large integers. Your professor requires exact answers, so the counts become increasingly large. Python handles this automatically, but arithmetic becomes

somewhat more expensive as the numbers grow.

The largest performance issues are likely the number of state-transition operations and memory usage.

4. What I would investigate first

Before changing your implementation, I would check these four things:

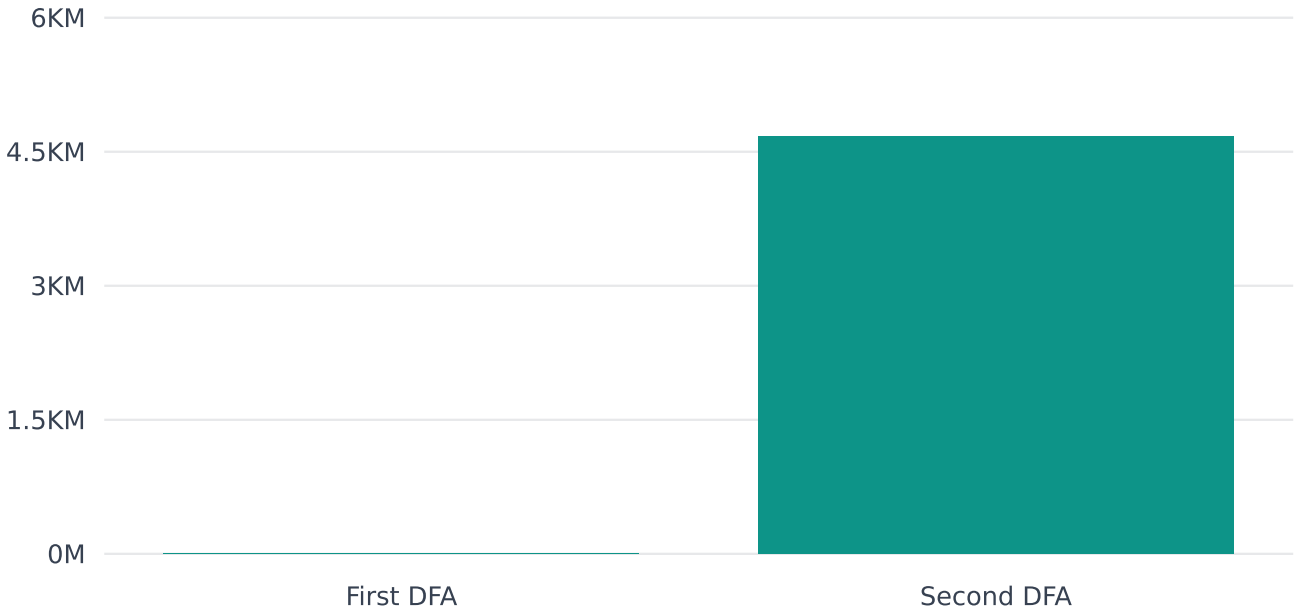
1. **Are you using 149 iterations for the second DFA when n = 300?** Using 300 would perform unnecessary work and produce the wrong count for the requested problem.
2. **Are you constructing the second DFA only once?** Rebuilding millions of states whenever you count would be wasteful.
3. **Are you storing every possible transition explicitly?** A large transition table can consume substantial memory, potentially causing swapping.
4. **Are you processing all possible product states?** Some may be unreachable, meaning your program could potentially avoid unnecessary work if the construction permits it.

The assignment requires a DFA with numerically labeled states and a counting algorithm using `prev` and `next`, so any optimization should still follow those requirements.

5. How much larger is the computational workload?

Approximate transition evaluations

For requested original string length n = 300. The second DFA uses 149 counting iterations.



This is why the first DFA might finish quickly while the second takes a very long time.

One question about your implementation

When you run the second DFA with n = 300, where does it seem to slow down?

- ○ Before asking for n (while constructing the DFA)
- ○ After entering n = 300 (during counting)
- ○ The program freezes or uses almost all available RAM
- ◉ I'm not sure

Continue troubleshooting →

The distinction matters because **slow DFA construction and slow dynamic programming require different optimizations**. We can investigate the cause without rewriting your code.

gpt-6-thinking

what should I do to speed up the seond DFA

Thinking

- **Optimizing the recurrence**

  I'm focusing on reducing each state's 16 transition combinations to two four-transition passes. I'm also checking how to reuse the existing two arrays safely, avoiding explicit product construction or an additional large temporary.

- **Reframing the computation**

  I'm noticing a much larger optimization: because both halves use the same DFA, I can count single-half endpoint totals once, then combine them through the boundary predicate. This may outperform pair-state dynamic programming dramatically, but it could depart from the assignment's required method.

- **Prioritizing speedups**

  I'm separating assignment-compliant optimizations from a faster mathematical shortcut. I'll emphasize compact integer states, precomputed single-DFA transitions, avoiding repeated allocations, and careful zero-state handling, while flagging any method that changes the required algorithm.

- **Balancing speed and compliance**

  I'm preserving the required DFA and recurrence, so I'll prioritize compact integer indexing, precomputed base transitions, and fewer allocations. I'm also considering a two-pass 4-plus-4 transition factorization, though it may need approval if the rubric requires literal copying.

- **Optimized and reframed the recurrence for faster compliant computation**

✓ Worked for 32s

For your CS 454 project, I would focus on **reducing the number of calculations and the amount of memory your second DFA uses**, rather than just trying to make individual operations faster.

There are several ways to improve performance, but some are more important than others.

1. Understand what's making it slow

Your second DFA uses a product construction, so it has approximately:

	First DFA	Second DFA
States	1,400	1,960,000
Input symbols	4	16
Transitions per iteration	5,600	31,360,000

For $n = 300$, your second DFA only needs to count paired strings of length:

$$300/2 - 1 = 149$$

But that still means approximately **4.67 billion transition evaluations** if you examine all 16 transitions for every product state in every iteration.

2. The optimizations I would consider

I would investigate these in order:

1. **Don't construct the second DFA repeatedly.** If you're rebuilding the product DFA every time the user enters a new value of n, that is unnecessary work.

2. **Avoid storing 31 million transitions if possible.** The second DFA's transitions can be determined from the first DFA's transitions. This could save considerable memory.

3. **Use integer state IDs instead of strings.** Representing states as numbers generally makes state lookups and array indexing more efficient.

4. **Avoid unnecessary calculations involving the fail state.** Since a fail state can never reach an accepting state, its counting value is always zero. You still need to handle transitions to it correctly.

5. **Make sure your counting algorithm runs 149 iterations for $n = 300$** rather than 300 iterations.

These are useful improvements, but there's an even more interesting optimization based on how your product DFA works.

3. Take advantage of the two-copy structure

Remember that one product DFA state contains two states from your original DFA:

$$(q_i, q_j)$$

An input symbol contains two characters:

$$(x, y)$$

The transition effectively updates both components:

$$\delta''((q_i, q_j), (x, y)) = (\delta(q_i, x), \delta(q_j, y))$$

Instead of thinking about all 16 combinations separately, notice that the two components can be processed independently.

Your counting recurrence is:

$$N_{i,j}(k+1) = \sum_{x \in \Sigma} \sum_{y \in \Sigma} N_{\delta(i,x),\delta(j,y)}(k)$$

You can mathematically split this into two stages:

- First, combine the four possibilities for the second DFA component.
- Then, combine the four possibilities for the first DFA component.

This is called **factoring the recurrence**.

Potential work reduction per iteration

Original approach	Factored approach
31.36M	**15.68M**
Transition contributions	Transition contributions

This could approximately halve the transition-processing work, although actual runtime improvement depends on your implementation.

One caution: your professor specifically requires a product DFA and the two-array counting method. Factoring the recurrence preserves the mathematics, but you should verify that this implementation approach meets the required algorithm format.

4. An even faster mathematical approach

There is another property of the second DFA worth understanding.

You're counting strings shaped like:

$$w = u \cdot aa \cdot v$$

where:

$$|u| = |v| = m - 1$$

Rather than counting every possible combination of two DFA states at every iteration, you could theoretically:

1. Count the valid left halves according to which original DFA state they finish in.
2. Independently count the right halves, reading them in reverse.
3. Determine which pairs of finishing states are compatible around the middle `aa`.
4. Combine the counts for compatible pairs.

This could reduce the counting work dramatically because you would perform most iterations over the original ~1,400-state DFA rather than the ~1.96-million-state product.

However, I would not immediately switch to this approach for your assignment. Your professor explicitly requires constructing L″ and applying the specified counting algorithm. This alternative is mathematically useful but may not satisfy those implementation requirements.

5. What I would do for your actual project

Before changing anything, determine whether your slowdown is caused by building the second DFA or by counting accepted strings.

If constructing the DFA is the problem, focus on how product states and their transitions are represented.

If counting is the problem, focus on reducing the work performed for every state during the 149 iterations.

One question

What programming language are you using for your DFA project — Python, Java, or C++?

That makes a significant difference in which optimization I'd recommend first. I can walk you through improving your current approach without writing the code for you.

gpt-6-thinking

Selected text:

Selection 1
Rather than counting every possible combination of two DFA states at every iteration, you could theoretically:

Count the valid left halves according to which original DFA state they finish in.

Independently count the right halves, reading them in reverse.

Determine which pairs of finishing states are compatible around the middle aa.

Combine the counts for compatible pairs.

This could reduce the counting work dramatically because you would perform most iterations over the original ~1,400-state DFA rather than the ~1.96-million-state product.

My request:
how would I apply this so it runs that much quicker

Thinking

- **Clarifying the faster method**

  I'm formulating the exact dynamic program: count each half by its DFA ending state, reverse the right half appropriately, then combine compatible state pairs. I'll verify boundary conditions, transitions, correctness, and complexity without writing code yet.

- **Checking boundary windows**

  I'm confirming that the join requires examining only windows touching the inserted two-character block. Up to seven windows matter, while shorter total strings can be handled directly; reversing the right half preserves its count distribution.

- **Confirming state counts**

  I'm verifying the state-space size: five-character buffers produce 1,365 contexts, plus a failure state. One forward pass counts each half, then compatibility checks combine reachable state pairs efficiently.

- **Addressing compliance**

  I'm noting an important constraint: if the project mandates the product DFA and two size-\(N\) arrays, this optimization should be presented as an optional acceleration unless the instructor approves it.

✓ Worked for 42s

You can make your second DFA counting problem **significantly faster by counting the left and right halves separately, then combining their results at the end**.

Instead of performing calculations over approximately 1.96 million product states for 149 iterations, you can perform most of the counting using your original ~1,400-state DFA.

I'll walk you through how to approach this without writing code.

One important warning: Your professor's PDF specifically requires constructing the second DFA (L″) and using the two-array counting algorithm. This optimization is mathematically valid for counting L′, but using it *instead of* the required product-DFA algorithm may not satisfy the assignment. I'll distinguish the two approaches below.

Step 1: Split the string into three parts

For the second problem, every string has the form:

$$w = u + aa + v$$

where u and v have the same length.

For $n = 300$:

$$|u| = |v| = 149$$

Left half (u)		Right half (v)
149 characters	**aa** Fixed	**149** characters

Rather than processing pairs of characters through the product DFA, consider counting how many valid halves end in each state of your original DFA.

Step 2: Count the left halves by their ending state

Your original DFA already recognizes whether a string satisfies the six-character window rule.

You can use it to count how many strings of length 149 end at each state.

For example, suppose a smaller DFA had these results:

Ending state	Number of strings reaching it
q_0	12
q_1	25
q_2	18

These are illustrative numbers.

Instead of keeping every individual string, you store just the number of strings reaching each state.

Call these counts:

$$A[i] = \text{number of length-149 strings reaching state } i$$

For this calculation, use **forward dynamic programming**:

1. Start with a count of 1 at the start state and 0 everywhere else.
2. For each state, distribute its current count across its four outgoing transitions.
3. Repeat for 149 characters.
4. Ignore the fail state when combining the final results.

This differs from the backward recurrence in your assignment, which starts with counts of 1 at accepting states. Both are valid counting methods, but your professor explicitly requires the backward recurrence.

Step 3: Count the right halves in reverse

For the right half, you want to know the characters closest to the middle `aa`.

You can accomplish this by processing the right half backward.

For example, if the right half is:

`bcabcd`

Read it as:

`dcbacb`

Your original DFA can still check whether it's valid because the six-character condition is unchanged when a string is reversed.

At the end, the DFA's buffer contains the five characters closest to the middle, in reverse order.

Here's the useful part: You actually don't need to perform the counting twice.

Every possible right-half string corresponds to exactly one reversed string. Both sides have the same length and use the same original DFA.

Therefore, their distributions of ending states are identical:

$$B[i] = A[i]$$

You can calculate the state counts once and reuse them for both halves.

Step 4: Determine which pairs of states are compatible

This is the most important part.

Just because the left and right halves are individually valid doesn't mean that joining them with `aa` produces a valid string.

You need to determine whether two ending states can be joined safely.

Suppose the left DFA ends with buffer:

`bcdab`

And the reversed-right DFA ends with buffer:

`cbacb`

Since the right side was processed backward, reverse that buffer to obtain `bcabc`.

Now combine them around `aa`:

b	c	d	a	b	a	a	b	c	a	b	c

Left buffer + aa + right buffer

Check every six-character window that touches the middle `aa`.

If they all contain a, b, c, and d, those two states are compatible.

If any window is missing a required letter, they're incompatible.

Define:

$$C[i][j] = \begin{cases} 1 & \text{if states } i, j \text{ are compatible} \\ 0 & \text{otherwise} \end{cases}$$

This is called a **compatibility matrix**.

You can determine compatibility for each pair of original-DFA states once, rather than repeating those checks for all 149 counting iterations.

Step 5: Combine the counts

Now suppose:

- 20 valid left halves reach state q_i.
- 30 valid reversed-right halves reach state q_j.
- These states are compatible.

How many complete strings can you create?

$$20 \times 30 = 600$$

Every one of those left halves can be combined with every one of those right halves, because compatibility depends on their ending states.

If the states are incompatible, they contribute zero.

The formula for the final answer is:

$$\boxed{\text{Answer}(300) = \sum_i \sum_j A[i] \times A[j] \times C[i][j]}$$

That's the entire idea!

You're multiplying the number of possible left halves by the number of compatible right halves and summing those combinations.

Step 6: How much faster could this be?

Using the approximate state counts from your assignment:

Estimated work for n = 300

Original product DFA counting	**4.67 billion**
Separate halves + combine	**~0.8M + 2M**

Approximately 0.8 million state-transition contributions, plus up to 2 million state-pair combinations. Compatibility checks require additional work.

This is a reduction of roughly three orders of magnitude in the main counting work. Actual speedup will depend on your implementation, memory use, and large-integer arithmetic.

Step 7: How I would organize the optimized approach

Original DFA (L)

~1,400 states, 4 input symbols

↓

Count strings of length 149

Store how many reach each DFA state

↓

Left counts

A[i]

Right counts

Reuse A[j]

↓

Check state compatibility

Using the five-character buffers and fixed middle aa

↓

Calculate the total

Sum A[i] × A[j] for compatible pairs

Step 8: Verify the idea with a smaller example

Before attempting $n = 300$, try $n = 6$.

The full string looks like:

$$u + aa + v$$

where each half contains two characters.

There are:

$$4^2 = 16$$

possible left halves and 16 possible right halves, giving 256 combinations.

Since the middle already contains `aa`, the remaining four characters must collectively contain b, c, and d.

Exactly **60 strings** meet that condition.

Therefore, your optimized method should return:

$$\boxed{\text{Answer}(6) = 60}$$

This gives you a useful small test case.

Important: How to use this in your actual assignment

Your PDF explicitly says the program must create the DFA for L″ and apply the specified two-array counting recurrence over its states.

The separate-halves approach avoids most of that product-state counting, so **it is an alternative algorithm, not necessarily an acceptable replacement for the one your professor requires.**

If you have to preserve the required algorithm, there's another optimization worth investigating: **factor the 16-symbol product transition into two groups of four transitions**. This still performs the recurrence over the product DFA's states but reduces repeated work.

My recommendation is to use the separate-halves method to understand the mathematical structure and independently verify your results, while keeping the

required product-DFA implementation for submission unless your professor approves the alternative.

The next thing I'd focus on is how your current second DFA stores its transitions. If you're storing an enormous transition table, you may be able to reduce both memory consumption and runtime without changing the required counting algorithm.

gpt-6-thinking